Chapter 2. Searching

Outline

- List Searches
- C++ Search Algorithms
- Hashed List Searches
- Collision Resolution

List Searches

- The algorithm used to search a list
  - depends on the structure of the list
- Two basic searches for arrays
  - the sequential search
  - the binary search
- Basic search concept is shown in Fig. 2-1

Basic Search Concept

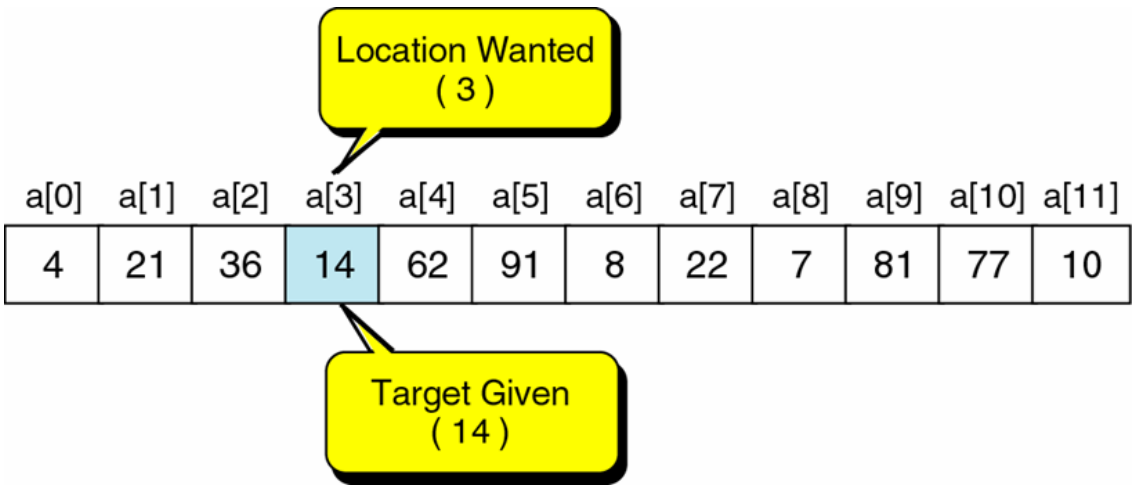


Figure 2-1 Locating data in unordered list

List Searches

- Sequential Search
  - Be used whenever the list is not ordered.
  - Use for
    - small lists
    - lists that are not searched often
- It give us two possibilities:
  - find it
  - or reach the end of the list

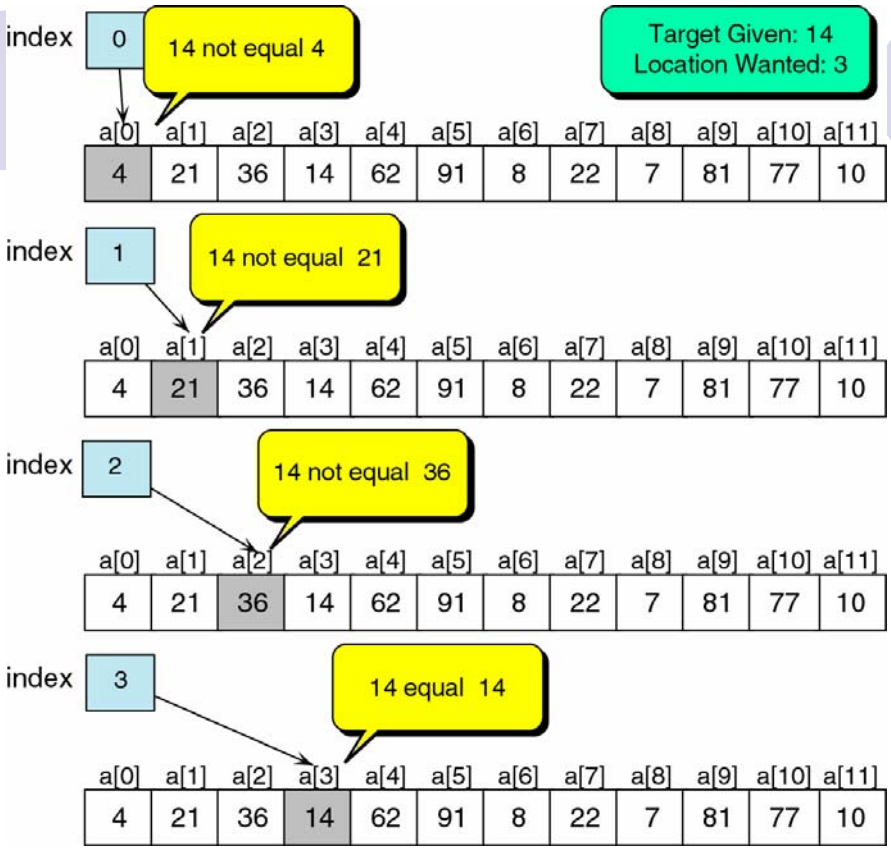
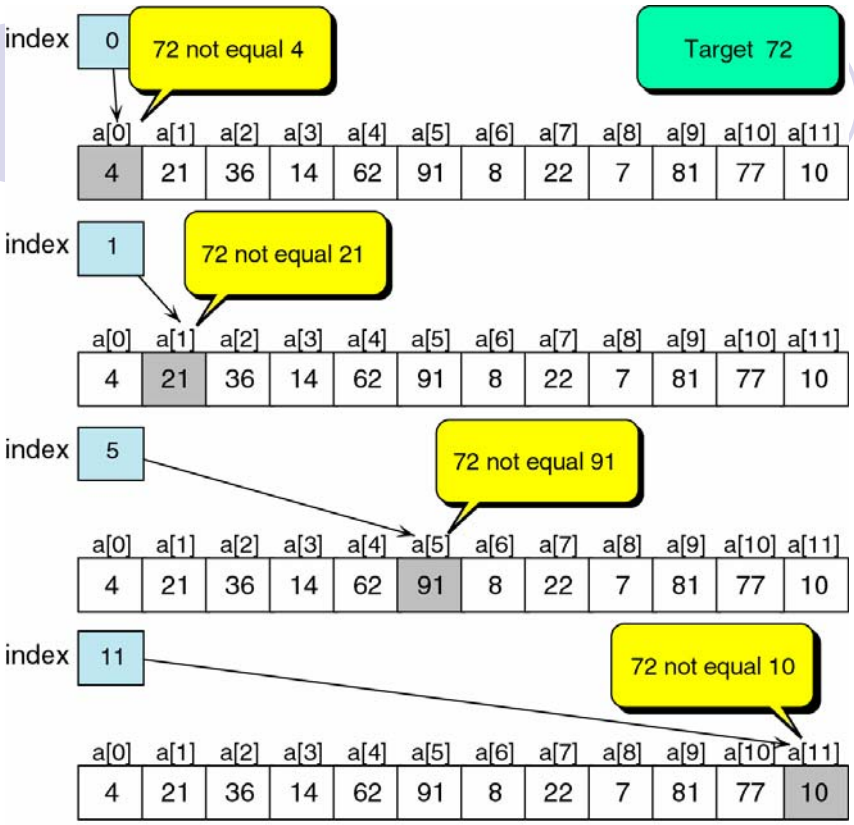


Figure 2-2 Search concept



Note: Not all test points are shown.

Figure 2-3 Unsuccessful search in unordered list

Sequential Search Algorithm

- Must tell the calling algorithm two things:
  - Did it find the target data？
  - At what index are the target data found
- requires four parameters:
  - the list
  - an index (to the last element)
  - the target
  - the address (where the target is found)

Sequential Search Algorithm

```
algorithm SeqSearch (val list   <array>,
                     val last   <index>,
                     val target <keyType>,
                     ref locn   <index>)

Locate the target in an unordered list of elements.
   Pre    list must contain at least one element
          last is index to last element in the list
          target contains the data to be located
          locn is address of index in calling algorithm
   Post   if found--matching index stored in locn & found true
          if not found--last stored in locn & found false
   Return found <Boolean>
 1  looker = 0
 2  loop (looker < last AND target not equal list[looker])
```

Algorithm 2-1 Sequential search

Sequential Search Algorithm

```
    1   looker = looker + 1
 3  end loop
 4  locn = looker
 5  if (target equal list[looker])
    1   found = true
 6  else
    1   found = false
 7  end if
 8  return found
end SeqSearch
```

Algorithm 2-1 Sequential search (continued)

Variations on Sequential Searches

- Three useful variations
  - the sentinel search
  - the probability search
  - the ordered list search

Variations on Sequential Searches

- Observation:
  - the loop tests two conditions:
    - end of list
    - target not found
- Knuth states:
  - “When the inner loop of a program tests two or more conditions, an attempt should be made to reduce it to just one condition.”

Variations on Sequential Searches

- **Sentinel search**
  - A target is put in the list
    - by adding an sentinel entry at the end of the array
    - And placing the target in the sentinel
- Pseudocode for the sentinel search is shown in Algorithm 2-2

Sentinel Search

```
algorithm SentinelSearch (val list    <array>,
                          val last    <index>,
                          val target  <keyType>,
                          ref locn    <index>)
Locate the target in an unordered list of elements.
   Pre    list must contain element at the end for the sentinel
          last is index to last data element in the list
          target contains the data to be located
          locn is address of index in calling algorithm
   Post   if found--matching index stored in locn & found true
          if not found--last stored in locn & found false
   Return found <Boolean>
```

Algorithm 2-2 Sentinel search

Sentinel Search

```
1  list[last + 1] = target
2  looker = 0
3  loop (target not equal list[looker])
   1   looker = looker + 1
4  end loop
5  if (looker <= last)
   1   found  = true
   2   locn   = looker
6  else
   1   found  = false
   2   locn   = last
7  end if
8  return found
end SentinelSearch
```

Algorithm 2-2 Sentinel search (continued)

Variations on Sequential Searches

- **Probability search**
  - the array is ordered with
    - the most probable search elements at the beginning
    - the least probable at the end
- It is especially useful when
  - few elements are the targets for most of the searches

Probability Search

- To ensure the probability ordering is correct over time
  - exchange the located element with the element before it in each search
- A typical implementation of the probability search is shown in Algorithm 2-3

Probability Search

```
algorithm ProbabilitySearch  (val list     <array>,
                              val last     <index>,
                              val target   <keyType>,
                              ref locn     <index>)

Locate the target in a list ordered by the probability of each
element being the target--most probable first, least probable last.
   Pre    list must contain at least one element
          last is index to last element in the list
          target contains the data to be located
          locn is address of index in calling algorithm
   Post   if found--matching index stored in locn & found true
             and element moved up in priority.
          if not found--last stored in locn & found false
   Return found <Boolean>
 1 looker = 0
 2 loop (looker < last AND target not equal list[looker])
   1   looker = looker + 1
 3 end loop
```


Algorithm 2-3 Probability search

Probability Search

```
 4  if (target equal list[looker])
    1   found = true
    2   if (looker > 0)
        1   temp              = list[looker - 1]
        2   list[looker - 1]  = list[looker]
        3   list[looker]      = temp
        4   looker = looker - 1
    3   end if
 5  else
    1   found = false
 6  end if
 7  locn = looker
 8  return found
end ProbabilitySearch
```

Algorithm 2-3 Probability search (continued)

Variations on Sequential Searches

- When searching a small list ordered
  - sequential search may be more efficient
    - Than binary search
- **Ordered list search**
  - not necessary to search to the end
  - stop searching when
    - target becomes less than or equal to current element

Ordered List Search

```
algorithm OrderedListSearch (val list   <array>,
                             val last   <index>,
                             val target <keyType>,
                             ref locn   <index>)

Locate target in a list ordered on target.
   Pre    list must contain at least one element
          last is index to last element in the list
          target contains the data to be located
          locn is address of index in calling algorithm
   Post   if found--matching index stored in locn & found true
          if not found--locn is index of first element > target
             or locn equal last & found is false
   Return found <Boolean>
```

Algorithm 2-4 Ordered list search

Ordered List Search

```
 1  if (target < list[last])
    1   looker = 0
    2   loop (target > list[looker])
        1   looker = looker + 1
    3   end loop
 2  else
    1   looker = last
 3  end if
 4  if (target equal list[looker])
    1   found = true
 5  else
    1   found = false
 6  end if
 7  locn = looker
 8  return found
end OrderedListSearch
```

Algorithm 2-4 Ordered list search (continued)

Binary Search

- A more efficient algorithm
  - whenever the list starts to become large
- Suggest:
  - consider binary searches whenever the list contains more than *16* elements
- Idea:
  - Testing the data in the element at the middle
    - determine if the target is in the first or second half of the list

Binary Search

  - Eliminate half the list until
    - we find the target or
    - the target is not in the list
- Three variables are required
  - the *beginning* of the list
  - the *middle* of the list
  - the *end* of the list

Target Found

- Figure 2-4 shows how we find 22
- Two conditions terminate binary search
  - the target is found
  - first becomes larger than last
- We can calculate mid as follows:

```
mid = (first + last)/2
```

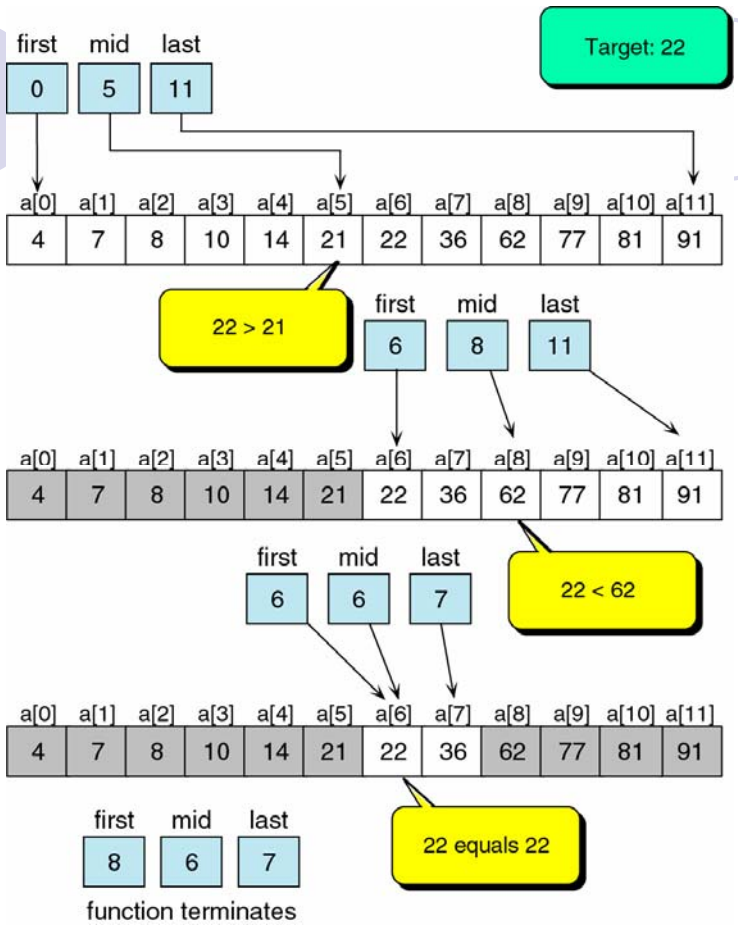



Figure 2-4 Binary search example

Target not found

- when the target is not in the list
  - Binary search algorithm stops
    - by testing for first and last crossing

Target not found

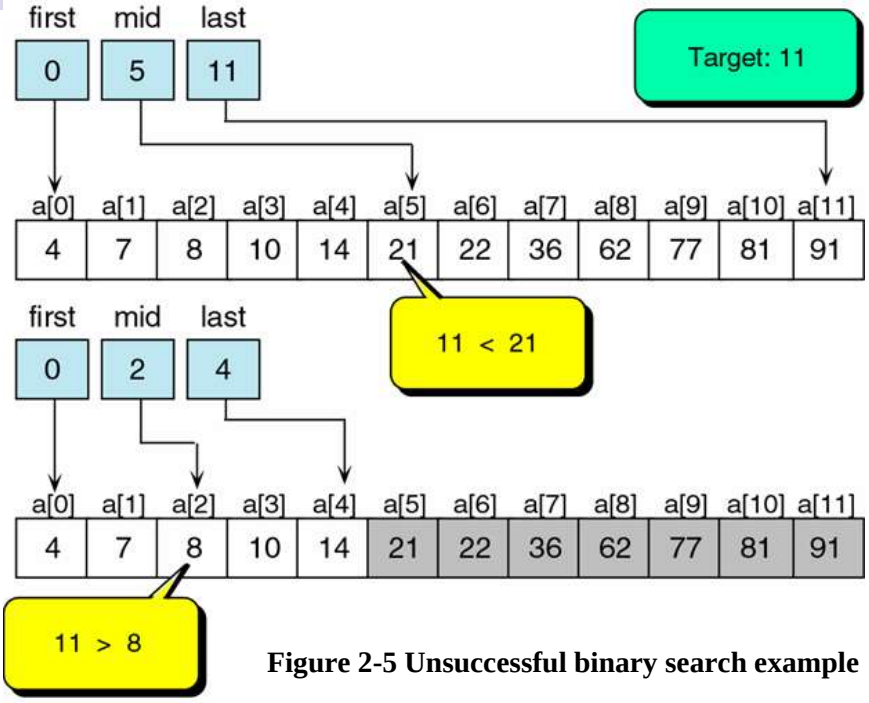


Figure 2-5 Unsuccessful binary search example

Target not found

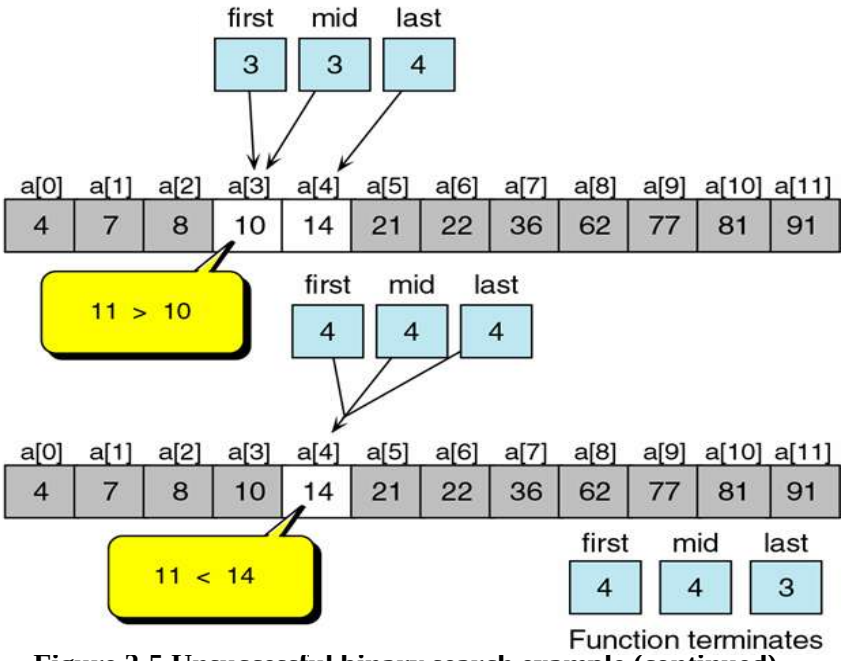


Figure 2-5 Unsuccessful binary search example (continued)

Binary search Algorithm

- Four parameters describe:
  - the list
  - the index of the last element
  - the target we are looking for
  - the address into which we place the located index

Binary search Algorithm

```
algorithm binarySearch (val list    <array>,
                        val end     <index>,
                        val target  <keyType>,
                        ref locn    <index>)
Search an ordered list using Binary Search
    Pre    list is ordered; it must contain at least one element
           end is index to the largest element in the list
           target is the value of element being sought
           locn is address of index in calling algorithm
    Post   FOUND: locn assigned index to target element
                  found set true
           NOT FOUND: locn = element below or above target
                      found set false
    Return found <Boolean>
 1  first = 0
 2  last = end
```

Algorithm 2-5 Binary search

Binary search Algorithm

```
 3  loop (first <= last)
    1   mid = (first + last) / 2
    2   if (target > list[mid])
        Look in upper half
        1   first = mid + 1
    3   else if (target < list[mid])
        Look in lower half
        1   last = mid - 1
    4   else
        Found equal: force exit
        1   first = last + 1
    5   end if
 4  end loop
```

Algorithm 2-5 Binary search (continued)

Binary search Algorithm

```
 5  locn = mid
 6  if (target equal list [mid])
    1   found = true
 7  else
    1   found = false
 8  end if
 9  return found
end binarySearch
```

Algorithm 2-5 Binary search (continued)

Analyzing Search Algorithms

- Which is the best?
  - Of the five search algorithm discussed
  - An application often determine which algorithm should be used
- Sequential Search
  - the basic loop for the sequential search:

```
while (looker < last && target != list[looker])
        looker++;
```

Analyzing Search Algorithms

- the sequential search algorithm is linear
  - its efficiency is O(n)

```
The efficiency of the sequential search is O(n).
```

- Binary Search
  - locate an item by repeatedly dividing the list in half

Analyzing Search Algorithms

- The loop for binary search

```
while (first <= last)
   {
    mid = (first + last) / 2;
    if (target > list[mid])
         first = mid + 1;
    else if (target < list[mid])
         last = mid - 1;
    else
         first = last + 1;
   } // while
```

Analyzing Search Algorithms

- The loop for binary search is a logarithmic loop

The efficiency of the binary search is $O(\log_2 n)$.

- Comparison
  - Disregarding the time required to order the list
    - Binary search is obviously more efficient for searching a list of any significant size
  - As shown in Table 2-1

Analyzing Search Algorithms

Size	Binary	Sequential (Average)	Sequential (Worst Case)
16	4	8	16
50	6	25	50
256	8	128	256
1000	10	500	1000
10,000	14	5000	10,000
100,000	17	50,000	100,000
1,000,000	20	500,000	1,000,000

Table 2-1 Comparison of binary and sequential searches

C++ Search Algorithms

- Sequential Search in C++
  - program 2-1
- program 2-1 Analysis
  - Why use *while* loop rather than *for* loop?
    - Finding something is an event, so use an event loop
  - Call-by-reference used for *locn*
    - Pass back the matching location


```
1  /* Locate the target in an unordered list of size elements.
2        Pre:    list must contain at least one item.
3                last is index to last element in the list
4                target contains the data to be located
5                locn is address to index in calling function
6        Post:   FOUND:     matching index stored at locn
7                           return true (found)
8                NOT FOUND: last stored in locn address.
9                           return false (not found)
10 */
11 bool seqSearch  (int  list[],
12                  int  last,
13                  int  target,
14                  int& locn)
15 {
16 // Local Definitions
17    int looker;
18
19 // Statements
20    looker = 0;
```

```
21      while (looker < last && target != list[looker])
22         looker++;
23
24      locn = looker;
25      return (target == list[looker]);
26   } //  seqSearch
```

Program 2-1 Sequential search

C++ Search Algorithms

- Binary Search in C++
  - shown in Program 2-2
- Program 2-2 Analysis
  - we use a *break* statement to force the *while* loop to end
  - we can force the loop to end with the following statement:

```
first = last + 1
```

```
/* Search an ordered list using Binary Search
      Pre:    list must contain at least one element
              end is index to the largest element in the list
              target is the value of element being sought
              locn is address to index in calling function
      Post:   FOUND: locn = index to target element
                     return true (found)
              NOT FOUND: locn = index below|above target
                     return false (not found)
*/
bool binarySearch  (int  list[],
                    int  end,
                    int  target,
                    int& locn)
{
// Local Definitions
   int first;
   int mid;
   int last;

// Statements
   first = 0;
   last  = end;
   while (first <= last)
      {
       mid = (first + last) / 2;
       if (target > list[mid])
          //  look in upper half
          first = mid + 1;
       else if (target < list[mid])
          //  look in lower half
          last = mid - 1;
       else
          //  found equal: force exit
          break;
      } //  end while
   locn = mid;
   return (target == list [mid]);
} //  binarySearch
```

Program 2-2 Binary search

Hashed List Searches

- The goal of a hashed search:
  - to find a data with only one test
- Basic concepts
  - the key determines the location of the data
    - Use a *hashing algorithm* to transform ***key*** into ***index***
  - Another way to describe:
    - A key-to-address transformation in which the keys map to addresses in a list
    - as shown in Figure 2-6

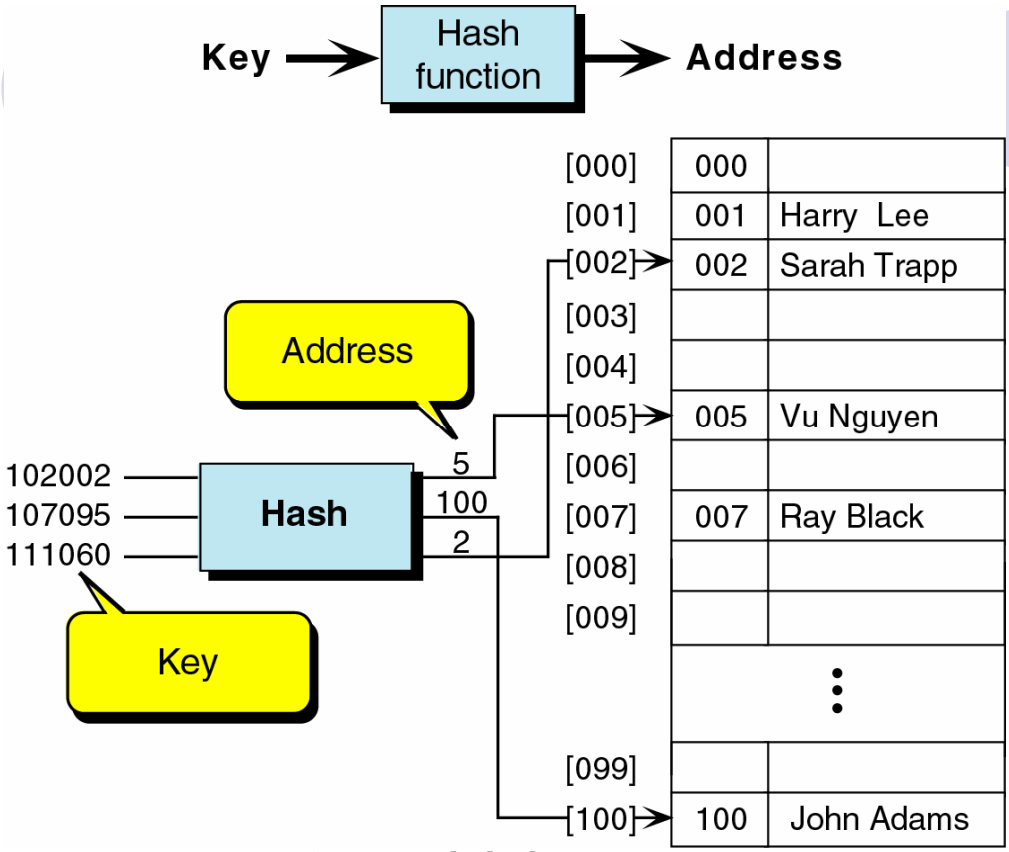


Figure 2-6 The hash concept

Basic Concept of Hash Search

- Synonyms
  - the set of keys that hash to the same location
- Collision
  - hashing algorithm produces an address for an insertion key
    - that address is already occupied

Basic Concept of Hash Search

- Home Address
  - the address produced by the hashing algorithm
- Prime Area
  - the memory that contains all of the home addresses
- When two keys collide at a home address
  - Resolve the collision
    - by placing the later data in another location
  - collision resolution concept is shown in Figure 2-7

Collision resolution concept

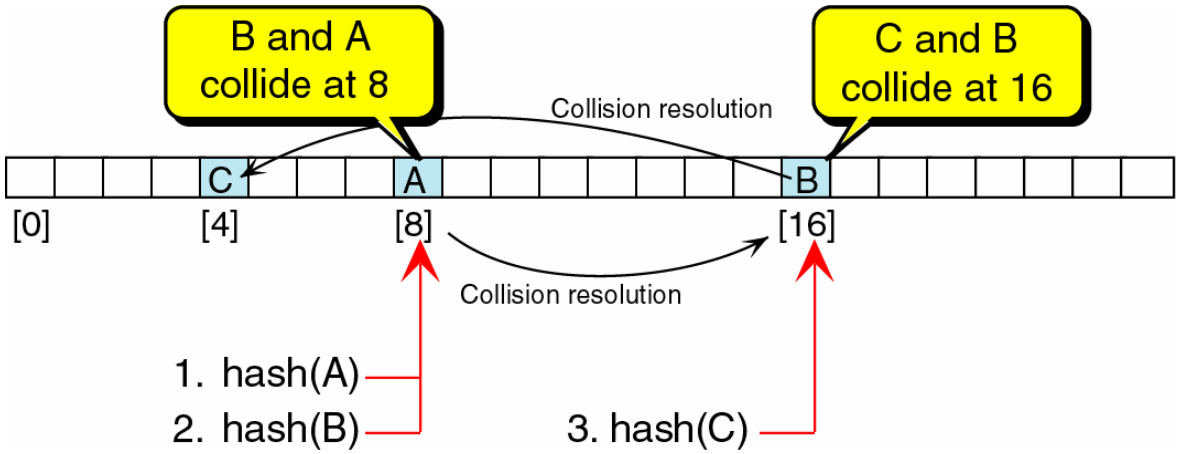


Figure 2-7 The collision resolution concept

Basic Concept of Hash Search

- Locate an element
  - use the same algorithm that we used to insert it
    - hash the key and check the home address
    - If it doesn't contain the desired element
      - Use the collision resolution algorithm to determine the next location
- Probe
  - Each calculation of an address and test for success

Hashing Method

- Basic hashing techniques

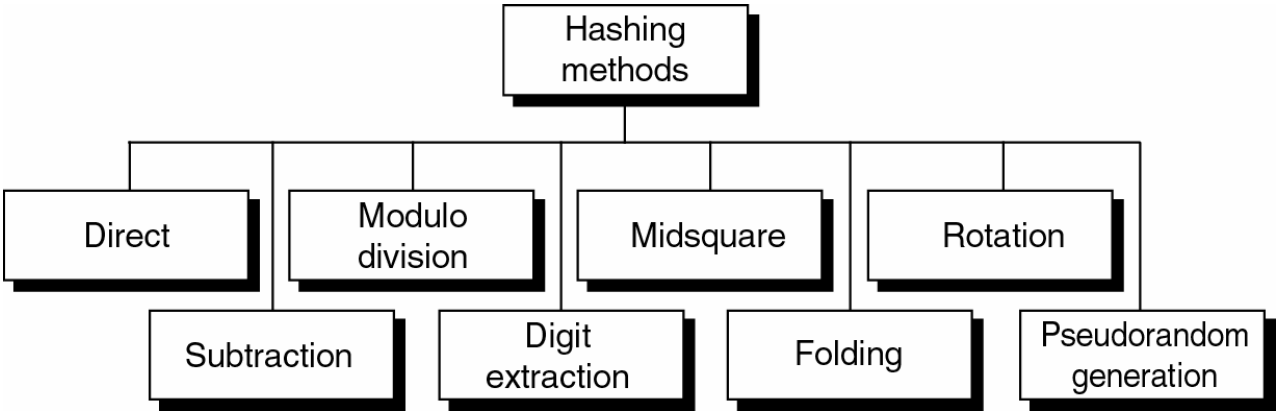


Figure 2-8 Basic hashing techniques

Direct Method

- Direct hashing
  - the key is the address
    - without any algorithmic manipulation
    - data structure must contain an element for every possible key
  - Evaluation
    - the situations you can use direct method is limited
    - but it is very powerful
      - it guarantees
        - No synonyms
        - No collisions

Direct hashing

- Application
  - Use the day of the month as the key
  - add the sale amount to the corresponding accumulator
    - The accumulation code is shown below

```
dailySales[sale.day] = dailySales[sale.day]
                     + sale.amount;
```

Direct hashing

- More complex example
  - A company has fewer than 100 employees
  - Let employee numbers be between 1 and 100
    - The employee number can be direct used
      - as the address of any individual record
    - The concept is shown in Figure 2-9

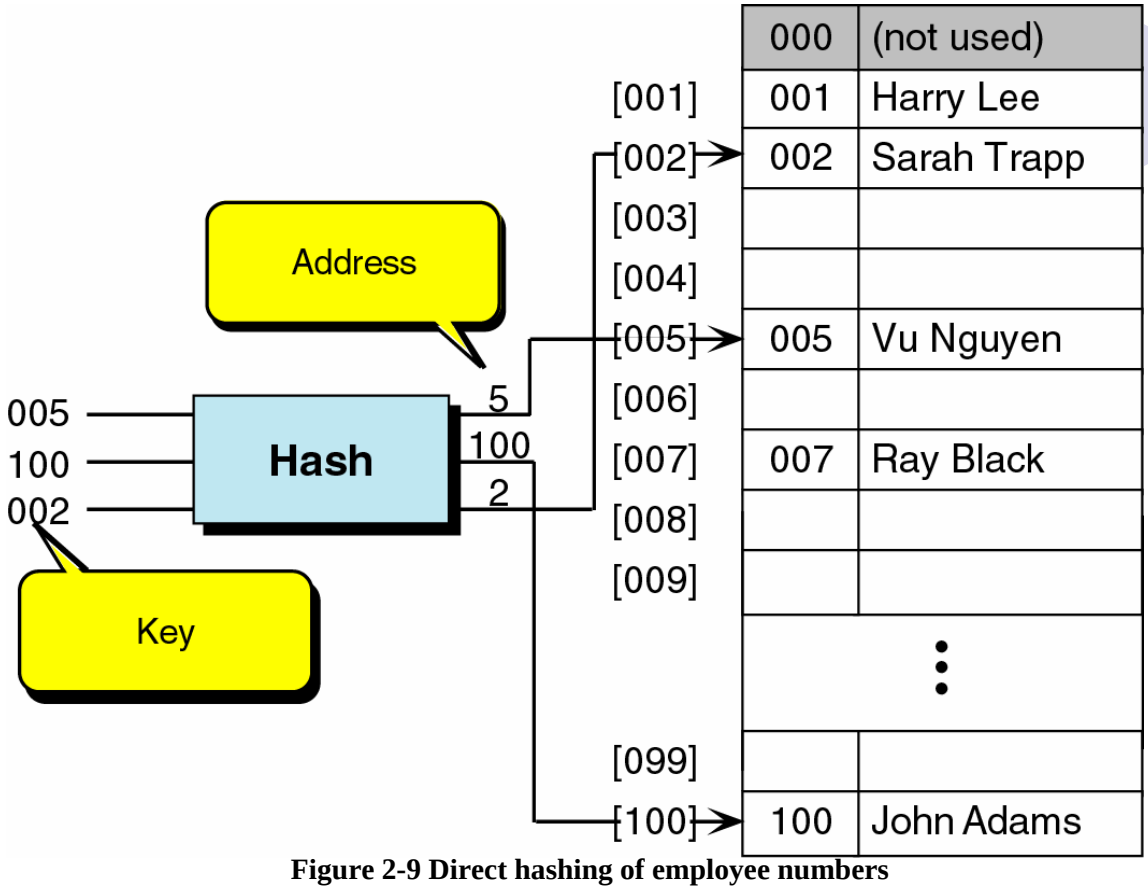


Figure 2-9 Direct hashing of employee numbers

Subtraction Method

- keys are consecutive but not start from 1
  - For example
    - the employee numbers are 1001~1100 (the company have 100 employees)
  - use a hashing function
    - subtracts 1000 from the key to determine address
- Only be used for small lists
  - in which the keys map to a densely filled list

Modulo-Division Method

- Also known as **division remainder**
  - divides the key by the array size
  - uses the remainder for the address

```
address = key MODULUS listSize
```

- This algorithm works with any list size
  - Prime number is better for the list size
    - Produce fewer collisions than other list size

Modulo-Division Method

- Application
  - A new employee numbering system
    - that will handle 1,000,000 employees
  - We provide data space for up to 300 employees
    - The first prime number greater than 300 is 307
    - E.g. Brian Devaux's employee number, 121267

```
121267/307 = 395 with remainder of 2
Therefore: hash(121267) = 2
```

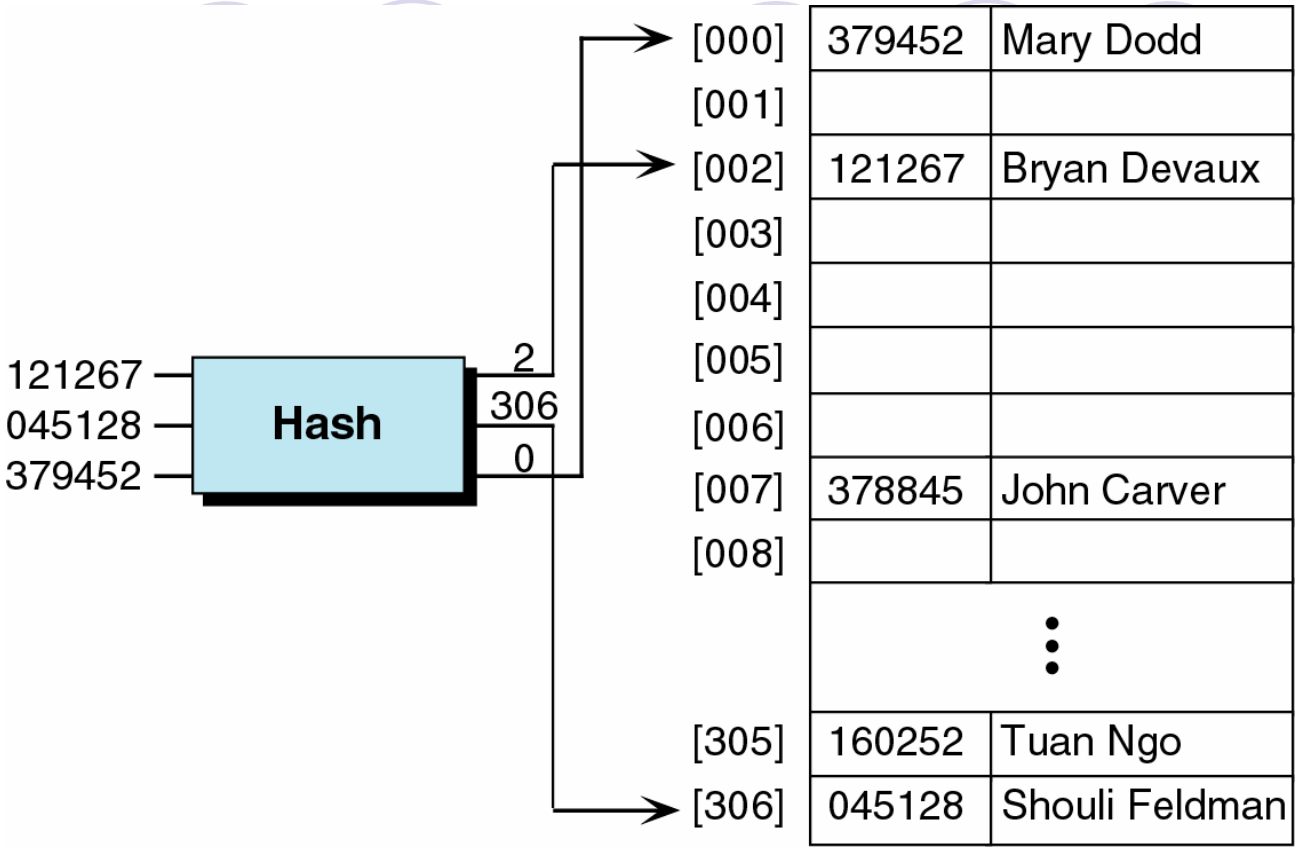
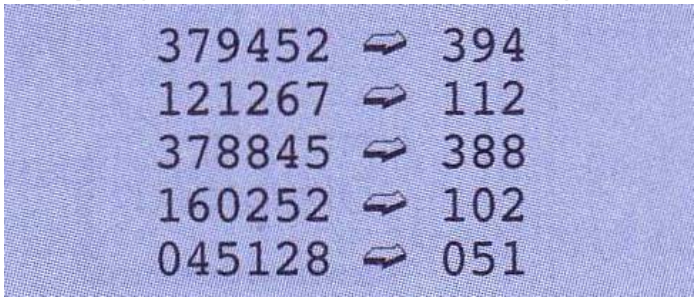



Figure 2-10 Modulo-division hashing

Digit-Extraction Method

- Digit extraction
  - selected digits are extracted from the key
    - and used as the address
- For example
  - 6-digit employee number to 3-digit address



Midsquare Method

- Midsquare hashing
  - the key is squared
  - the address selected from the middle of the squared number
- The most obvious limitation of this method
  - the size of the key
  - e.g: the midsquare address calculation of 9452

```
9452 * 9452 = 89340304: address is 3403
```

Midsquare Method

- Variation of Midsquare Method
  - select a portion of the key
  - for example:

```
379452: 379 * 379 = 143641 ⇨ 364
121267: 121 * 121 = 014641 ⇨ 464
378845: 378 * 378 = 142884 ⇨ 288
160252: 160 * 160 = 025600 ⇨ 560
045128: 045 * 045 = 002025 ⇨ 202
```

  - the same digits must be selected from the product

Folding Method

- Two folding methods:
  - fold shift
  - fold boundary
- Fold shift
  - the key value is divided into parts
    - whose size matches the size of the required address
  - the left and right parts are shifted and added with the middle part
  - Shown in Figure 2-11(a)

Folding Method

- Fold boundary
  - The left and right numbers are folded on
    - a the boundary between them and the center number
  - Shown in Figure 2-11(b)

- The two folding methods give different hashed addresses

Folding Method

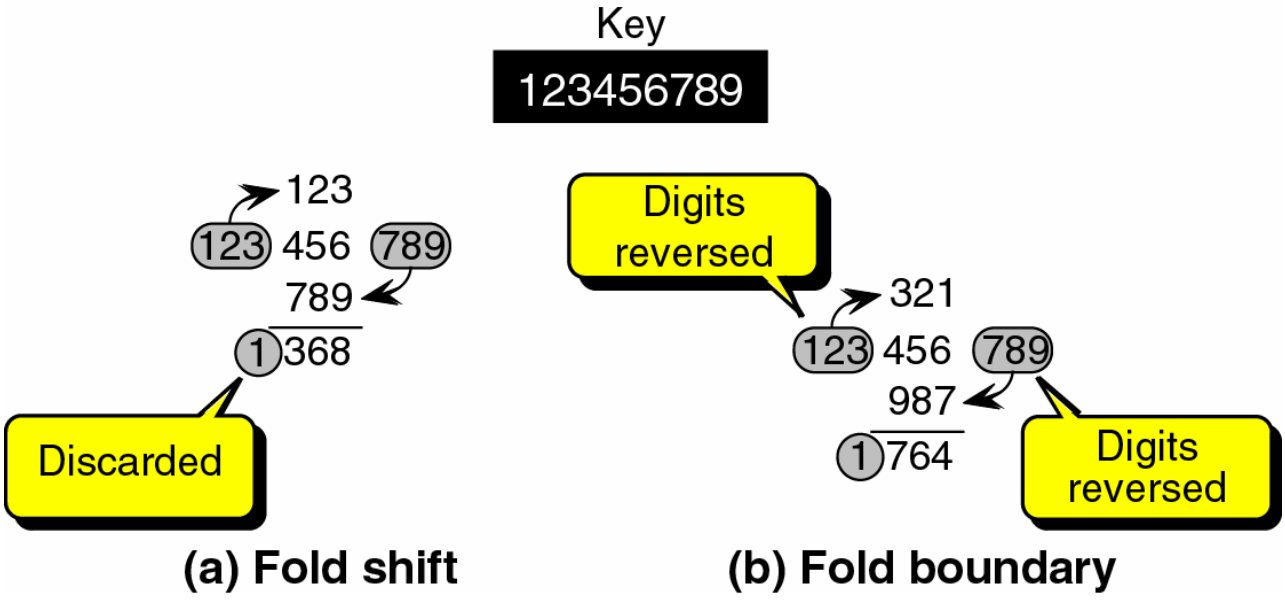


Figure 2-11 Hash fold examples

Rotation Method

- Rotation hashing
  - usually work with other hashing methods
  - is most useful when keys are serial
- Example
  - If hashing keys are identical
    - except the last character
  - Resolution:
    - Rotating the last character to the front of the key

Rotation Method

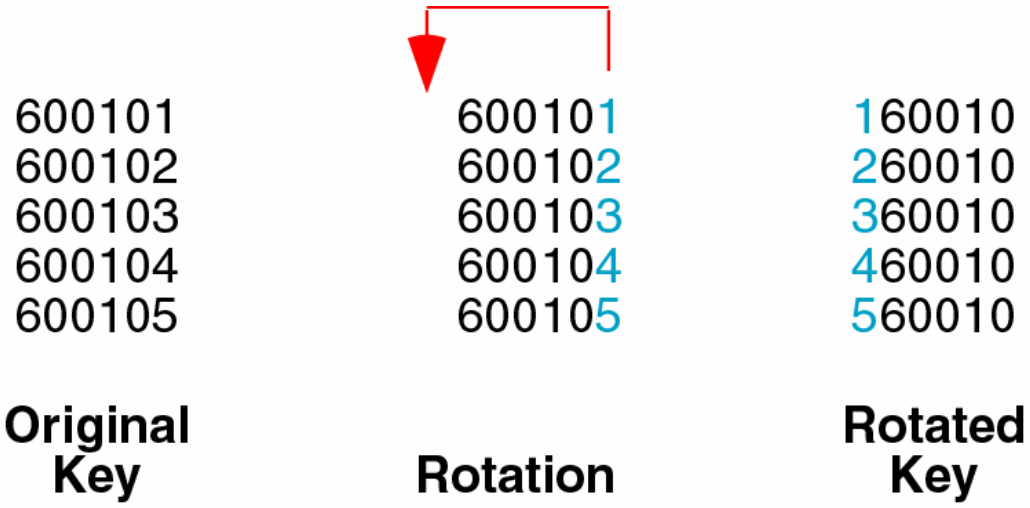


Figure 2-12 Rotation hashing

Rotation Method

- In Figure 2-12
  - all keys now end in 60010
  - modulo division would not work well
- Rotation is often used in combination with
  - ***folding*/*pseudorandom*** hashing
  - By **fold shift** to two-digit address
    - We get 26, 36, 46, 56, 66
    - Spreading the data more evenly

Pseudorandom Method

- Pseudorandom number generator
  - the key is used as the seed
- Scale the resulting random number
  - into the possible address range
  - by using *modulo division*
- A common RNG is as follows:

$$y = ax + c$$

Pseudorandom Method

- The result is divided by the list size
  - with the remainder being the hashed address
- For maximum efficiency
  - The factor **a** and **c** should be prime numbers

Pseudorandom Method

- Example from Figure 2-10
  - Use a=17, c=7

```
y = ((17 * 121267) + 7) modulo 307
y = (2061539 + 7) modulo 307
y = 2061546 modulo 307
y = 41
```

Hashing Algorithm

- hashing a key to an address
  - the hashing methods may work well
- hashing a large files is more complex
  - requires analysis of the population of keys
    - determine the number of synonyms
  - An example is discussed below

Algorithm 2-6 A hashing algorithm

Hashing Algorithm

- Assume
  - An alphanumeric key up to 30 bytes
  - Need to hash into 32-bit address
- Steps
  - Convert the key into a number by ASCII
  - Accumulate the gained values
  - Rotate the bits in the address
  - Take the absolute value of the address

Hashing Algorithm

```
algorithm hash  (val key      <array of characters>,
                 val size     <integer>,
                 val maxAddr  <integer>,
                 ref addr     <integer>)
This algorithm converts an alphanumeric key of size characters
into an integral address.
   Pre    key is a key to be hashed
          size is the number of characters in the key
          maxAddr is the maximum possible address for the list
   Post   addr contains the hashed address
 1 looper = 0
 2 addr   = 0
```

Algorithm 2-6 A hashing algorithm


```
Hash key
 3 loop (looper < size)
   1   if (key[looper] not space)
       1   addr = addr + key[looper]
       2   rotate addr 12 bits right
   2   end if
   3   looper = looper + 1
 4 end loop

Test for negative address
 5 if (addr < 0)
   1   addr = absolute(addr)
 6 end if
 7 addr = addr modulo maxAddr
 8 return
end hash
```

Algorithm 2-6 A hashing algorithm (continued)

Algorithm 2-6 Analysis

- Discussion in Algorithm 2-6
  - The rotation in statement 3.1.2
    - can be finished by an assembly language instruction
  - This algorithm uses three of the hashing methods
    - Fold shift
    - Rotation
    - Modulo division

Collision Resolution

- No hashing methods are 1-to-1 mapping
  - except the direct and subtraction methods
  -
- Several collision resolution methods
  - They are independent of the hashing algorithm
  - shown in Figure 2-13

Collision Resolution

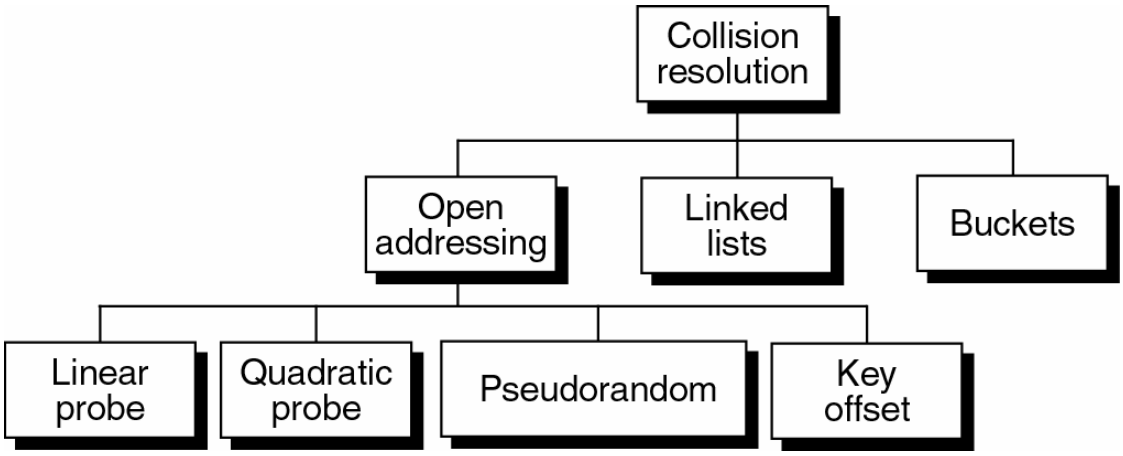


Figure 2-13 Collision resolution methods

Collision Resolution — Three concepts

- **load factor** of a hashed list

$$\alpha = \frac{k}{n} \times 100$$

  - The formula in which
    - *k*: the number of filled elements
    - *n*: the total number of elements

Collision Resolution — Three concepts

- Clustering
  - the tendency of data to build up unevenly across a hashed list
  - If the list contain a high degree of clustering
    - The number of probes to locate an element grows
    - Reduces the processing efficiency of the list

Collision Resolution — Three concepts

- Two types of Clustering
  - ***Primary clustering***
    - occurs when data cluster around a home address
    - easy to identify
  - ***Secondary clustering***
    - occurs when data become grouped along a collision path throughout a list
    - is not easy to identify
- Goal
  - Design hashing algorithm to minimize clustering

Collision Resolution — Three concepts

- Final concept
  - Number of elements examined for a place to store the data must be limited
  - Traditional limits of examining all elements
    - The search is not sequential
      - Finding the end not mean every element is tested
    - Examining every element would be time-consuming
    - Some of the collision resolution techniques cannot physically examine all of the elements

Collision Resolution

- Open Addressing
  - Resolve collisions in the prime area
    - That contains all of the home addresses
  - Four different methods:
    - linear probe
    - quadratic probe
    - double hashing
    - key offset

Open Addressing

- Linear Probe
  - If data cannot be stored in the home address
    - adding 1 to the current address.
  - Two advantages
    - quite simple to implement
    - data tend to remain near their home address
  - Tend to produce primary clustering

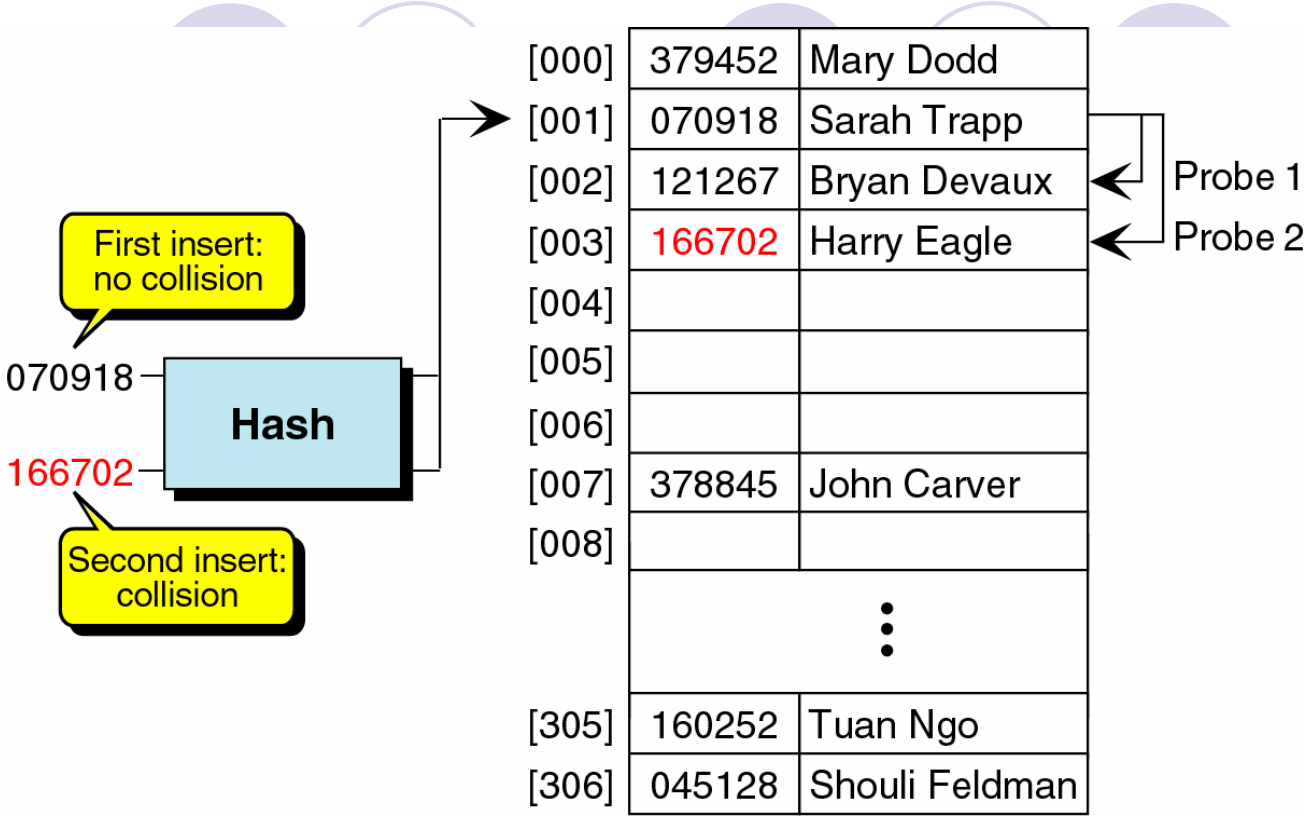


Figure 2-14 Linear probe collision resolution

Open Addressing

- Quadratic Probe
  - increment is square of collision probe number
  - potential disadvantage
    - The time to compute the square
  - limitation
    - Can not generate a new address for every element
      - If the list size=100, see Table 2-2
        - Only 59 of the probes generate unique addresses
        - The other 41 locations will not be probed

Quadratic Probe

Probe Number	Collision Location	Probe^2 and Increment	New Address
1	1	$1^2 = 1$	1 + 1 ⇨ 02
2	2	$2^2 = 4$	2 + 4 ⇨ 06
3	6	$3^2 = 9$	6 + 9 ⇨ 15
4	15	$4^2 = 16$	15 + 16 ⇨ 31
5	31	$5^2 = 25$	31 + 25 ⇨ 56
6	56	$6^2 = 36$	56 + 36 ⇨ 92
7	92	$7^2 = 49$	92 + 49 ⇨ 41
8	41	$8^2 = 64$	41 + 64 ⇨ 5
9	5	$9^2 = 81$	5 + 81 ⇨ 86
10	86	$10^2 = 100$	86 + 100 ⇨ 86

Table 2-2 Quadratic collision resolution increments

Open Addressing

- Pseudorandom Collision Resolution
  - use the collision address as a factor
    - in the random number calculation
  - Significant limitation
    - All keys follow only one collision resolution path

```
y = (ax + c) modulo listSize
  = (3 × 1 + 5) Modulo 307
  = 8
```

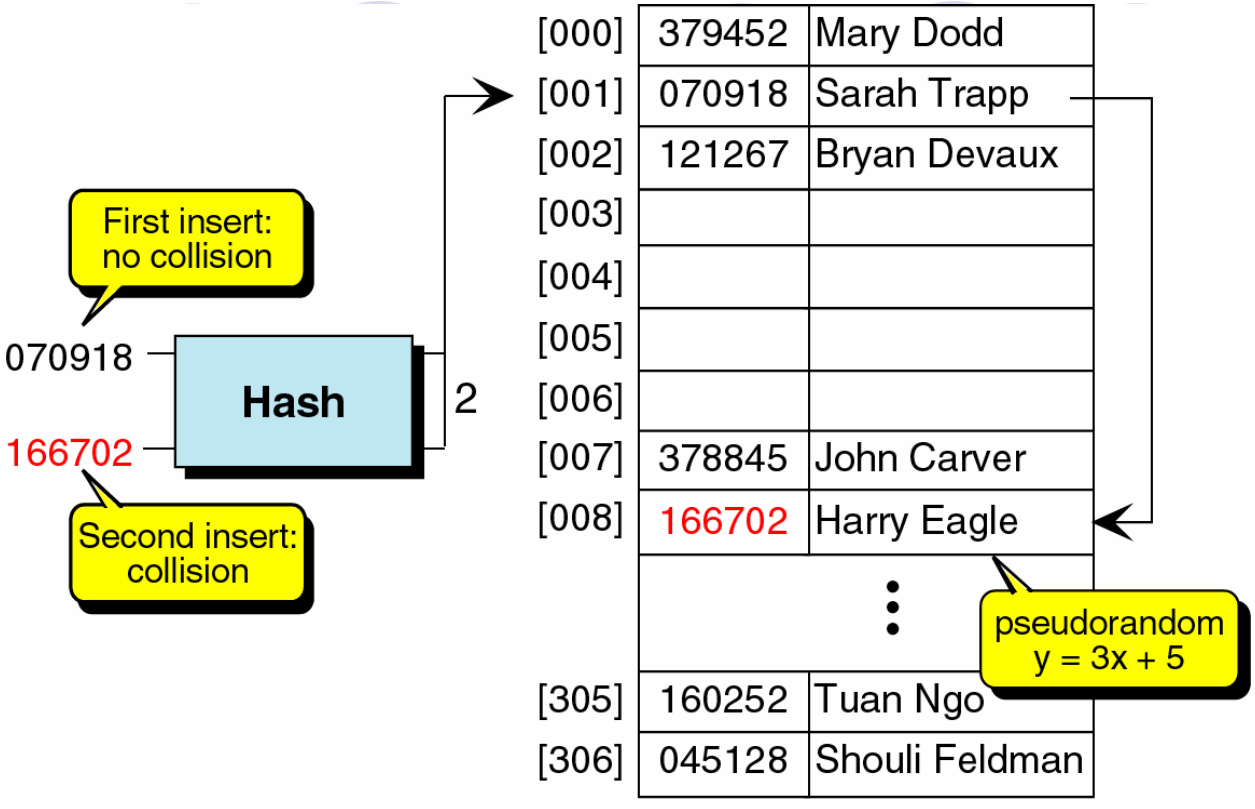



Figure 2-15 Pseudorandom collision resolution

Open Addressing

- Key Offset
  - a double hashing method
    - produces different collision paths for different keys
  - one of the simplest versions
    - simply adds the quotient of the key divided by the list size to the address

```
offSet     = ⌊key/listSize⌋
address    = ((offSet + old address) modulo listSize)
```


Key Offset

- For example
  - The key is 070918 and the list size is 307
    - using the modulo-division hashing method
      - generate an address of 1
  - Shown in Figure 2-15
    - 166702 produce a collision at address 1
    - Using key offset to create the next address

```
offSet     = ⌊166702/307⌋ = 543
address    = ((543 + 001) modulo 307) = 237
```

Key Offset

- To really see the effect of key offset
  - We need to calculate several different keys
    - all of them hash to the same home address 1

Key	Home Address	Key Offset	Probe 1	Probe 2
166702	1	543	237	166
572556	1	1865	024	047
067234	1	219	220	132

Table 2-3 Key-Offset examples

Linked List Resolution

- A major disadvantage to open addressing
  - each collision resolution increases the probability of future collision
- Linked list
  - an ordered collection of data
    - each element contains the location of next element

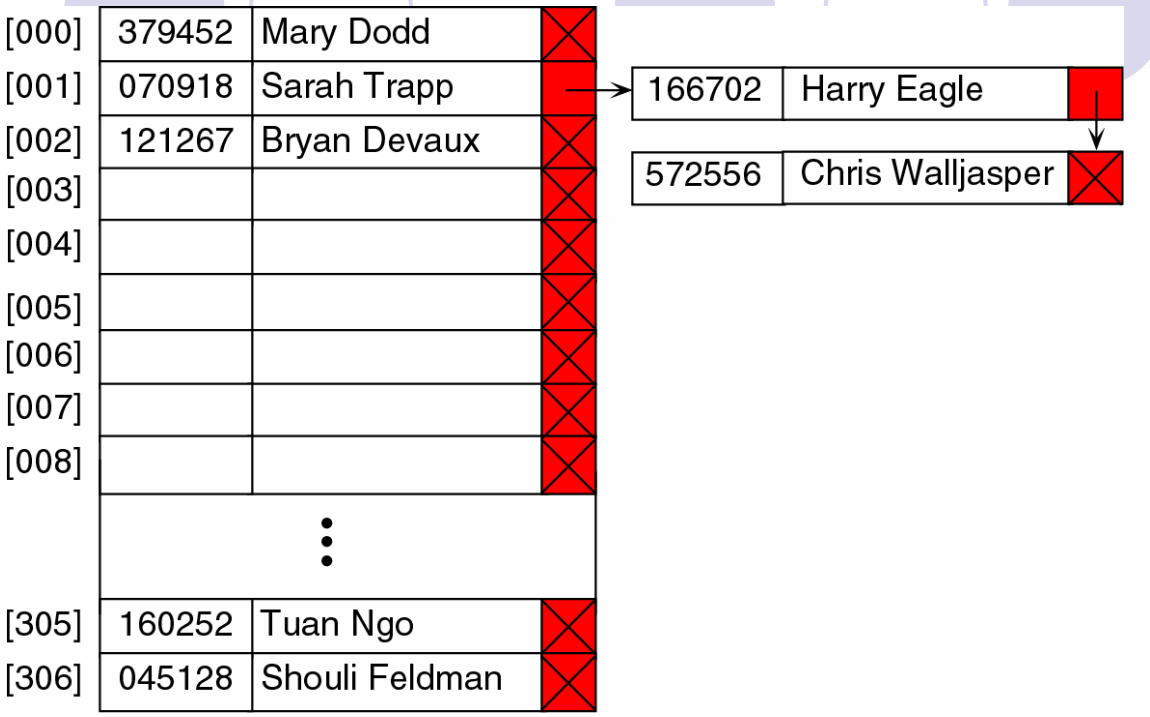


Figure 2-16 Linked list collision resolution

Linked List Resolution

- uses two storage areas
  - the prime area
  - the overflow area
- linked list data can be stored in any order
- Most common order
  - a last in-first out (LIFO) sequence
    - element is placed at the beginning of the overflow list
  - a key sequence

Bucket Hashing

- Because a bucket can hold multiple data
  - collision are postponed until the bucket is full
- Two problems
  - it uses significant more space
  - it dose not completely resolve the collision problem

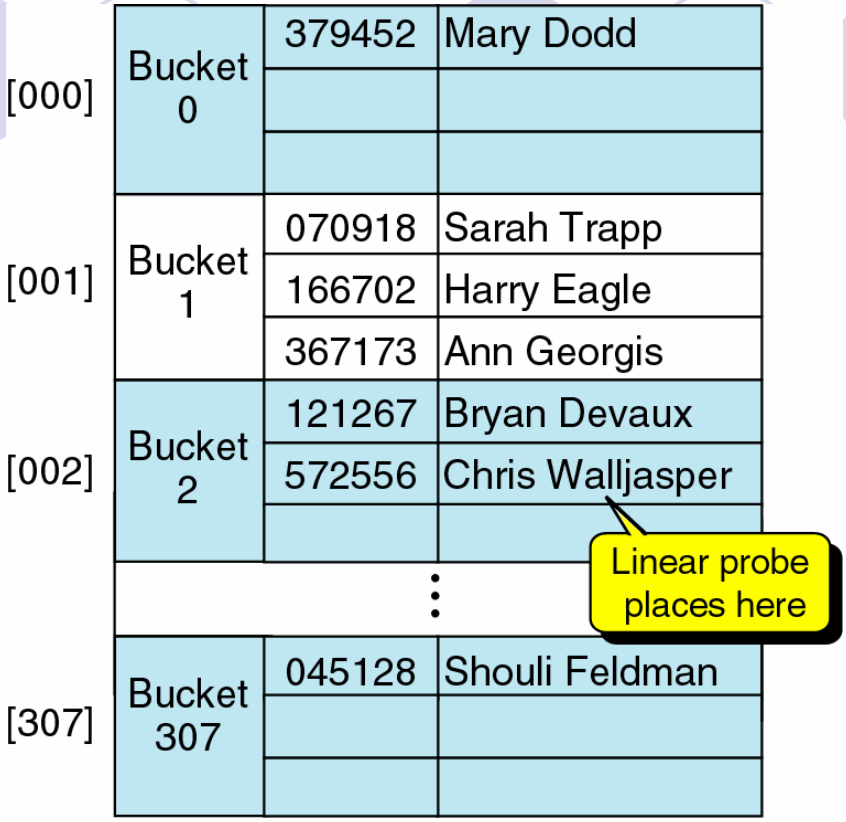


Figure 2-17 Bucket hashing

Combination Approaches

- A complex implementation
  - often uses multiple steps
- For example
  - One large database hashes to a bucket
    - If the bucket is full
      - It use a set number of linear probes
      - then uses a linked list overflow area